

Lab 7: Pointers

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** Address-of, dereference, pass-by-reference, pointers and arrays

Objectives

- Use `&` to take an address and `*` to dereference a pointer.
- Modify a caller's variable from inside a function via a pointer parameter.
- Understand the relationship between arrays and pointers (`a[i] = *(a+i)`).
- Recognize common pointer pitfalls: `NULL`, uninitialized pointers, wrong type.

Quick Reference

A pointer holds an address. You can read this as “*p points to x*”.

```
int x = 5;
int *p = &x;          /* p holds the address of x -- "p points to x" */
*p = 10;              /* write through p; now x == 10 */
printf("%d\n", *p);   /* read through p; prints 10 */
```

Pass-by-reference. To let a function modify a caller's variable, pass its address; the function declares a pointer parameter.

```
void inc(int *p) { (*p)++; }
int main(void) { int a = 5; inc(&a); /* a == 6 */ }
```

Arrays and pointers. For `int a[5]`; the name `a` *decays* to `&a[0]` in most contexts. So `a[i] ≡ *(a+i) ≡ *(&a[0]+i)`. Pointer arithmetic is in *elements*, not bytes: `p + 1` skips one `int`.

Pitfalls. `int *p;` alone is uninitialized – dereferencing it is undefined behavior. Always set it (`p = &x;`) or assign `NULL` and check before use.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Swap via Pointers (20 min)

Write `void swap(int *a, int *b)` that swaps the integers its arguments point to. Read two integers in `main`, call `swap(&x, &y)`, and print the result. Why couldn't you write this function with plain `int` parameters? Comment that in your code.

In-Lab Exercise 2: Min/Max via Out-Parameters (30 min)

Write `void min_max(int a[], int n, int *out_min, int *out_max)` that finds the minimum and maximum of the array and writes them through the pointer parameters. *One* function, *two* results – pointers are how C returns multiple values.

In-Lab Exercise 3: Pointer Walk over an Array (35 min)

Read `N` doubles into an array and compute the sum using a pointer `p` instead of an index: start with `p = a`, end at `a + N`, increment with `p++`, accumulate `*p`. Print the sum. Compare your code with the index-based version mentally – they should produce the same numbers.

In-Lab Exercise 4: In-Place Reverse with Two Pointers (35 min)

Write `void reverse(int *a, int n)` that reverses an array in place using two pointers: `left = a`, `right = a + n - 1`. Swap and walk them toward each other until they meet. Reuse `swap` from Exercise 1.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Address-of and dereference

What does this print?

```
#include <stdio.h>
int main(void) {
    int a = 3, b = 7;
    int *p = &a;
    *p = b;
    b = *p + 1;
    printf("%d %d\n", a, b);
    return 0;
}
```

Trace & Predict 2: Array via pointer arithmetic

Predict the exact output:

```
#include <stdio.h>
int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a + 1;
    printf("%d %d %d\n", *p, *(p + 2), p[-1]);
    return 0;
}
```

Find the Bug 1: Pass-by-value when pointers were needed

Why doesn't this swap? Fix it.

```
#include <stdio.h>
void swap(int a, int b) { int t = a; a = b; b = t; }
int main(void) {
    int x = 1, y = 2;
    swap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Write Code on Paper 1: Divide with remainder via pointers

Write `void divmod(int a, int b, int *q, int *r)` that stores a/b in `*q` and $a\%b$ in `*r`. Demonstrate from `main`. You may assume $b \neq 0$.

Write Code on Paper 2: Stats via a single pointer pass

Write `void stats(const int *a, int n, int *sum, int *positive_count)` that walks `a` with a pointer and writes both the total sum and the count of strictly positive elements through the out-parameters. Demonstrate from `main`.

Heads up. Pointers reappear in every remaining lab – strings (Lab 8), structs (Lab 9), and files (Lab 10). Pointers are **1/3** of the final exam, so do the take-home in full.